

Structured bindings with explicit types

Abstract

Here's the tl;dr: allow

```
auto [std::string a, int b] = f();
```

Rationale

C++ allows explicit and non-deduced typing to be used at API boundaries when that's deemed the right choice. This is one of the biggest strengths of C++; when users opt in to an explicit type, incompatible API changes will break code loudly, and a compiler will tell a user what places to fix when an API refactoring occurs.

Structured bindings don't allow such an explicit typing to occur; yet it seems very plausible that sometimes structured bindings are used at such API boundaries. Therefore we should allow making the types of the bindings concrete and explicit, not just deduced.

"But bindings must be bindings, not new variables..."

They are. The rule must be that, regardless of whether we allow any conversions, that new objects are never created by the bindings.

"Conversions?"

Well. For the most important use cases, being able to specify the exact type of a binding is already a big help. However, it seems like we can and perhaps should allow some conversions, like derived-to-base and qualification conversions. That is,

```
struct B {}; struct D : B {};
struct X {D a; D b;};
auto [D x, D y] = X(); // this should be relatively non-controversial
auto [B x, B y] = X(); // this seems reasonable
auto [const D x, const D y] = X(); // this seems very reasonable
auto [const B x, const B y] = X(); // this seems reasonable
```

As background material, Gabriel Dos Reis recommended looking at [N1782](#), especially sections 6.7 and 7.*.

"If we allow conversions, what's the decltype of the binding?"

In the above example, if the type of the thing to bind is D and we convert to B, I'd expect the decltype of the binding to be B. If the type of the thing to bind is D& and we convert to B, I'd expect the decltype of the binding to be B&. That is, convert the non-reference type (again, without creating a new object), and then reflect the tuple_element's type (reference or non-reference) in the result.

"Has this been implemented?"

No.

"Does this need to go into C++20?"

Yes. The sooner the better. The current forcing of using deduction and only deduction with structured bindings causes long-term harm, because it prevents writing concretely-typed API boundaries from the client side of structured bindings. Separate static_asserts to assert a type of a binding are antithetical.

I did say that I need to implement this proposal before it becomes C++20 material. That implementation is pending, but I am

going to propose this anyway so that it has a chance to get in.

"Should it be possible to specify the type of the whole-object?"

As in "ConcreteType [a, b] = f()", that doesn't seem to be necessary. If I use structured bindings, the types of the bindings are what matters in a strong/concrete API boundary, the whole-object type is irrelevant. If I don't use structured bindings, I can already use concrete types.

"Should lambda captures also allow specifying types?"

As in "[std::string a = f()]{...}" or even "[std::string x]{...}", well, that would probably be useful. If this proposal is accepted, lambdas are the last place where a concrete type cannot be specified. The snag there is that a by-value capture will create a new object, so it allows more conversions. The syntaxes end up looking the same but doing different things (which is not new, though, the current syntax of lambda captures and structured bindings looks the same but does vastly different things).